

# Python p/ Processamento de Dados

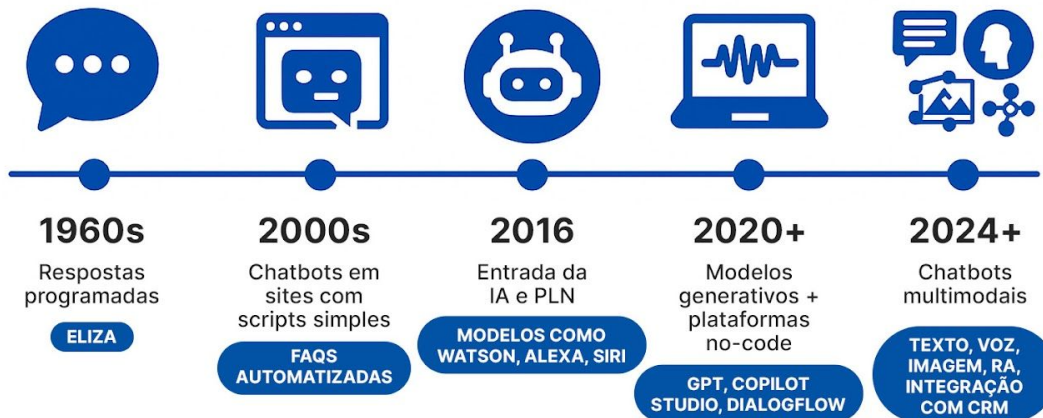
Etapa 9

Professor(a):

E-mail: [marcelo.hama@prof.infnet.edu.br](mailto:marcelo.hama@prof.infnet.edu.br)

# Criação de Serviços e Integração com IA

## Linha do Tempo: A Evolução dos Chatbots



Fonte: <https://bluecx.com.br/2025/06/17/evolucao-dos-chatbots/>

# Criação de Serviços e Integração com IA

## PARTE 1

### **Interação com IA em OpenAI e Gemini**

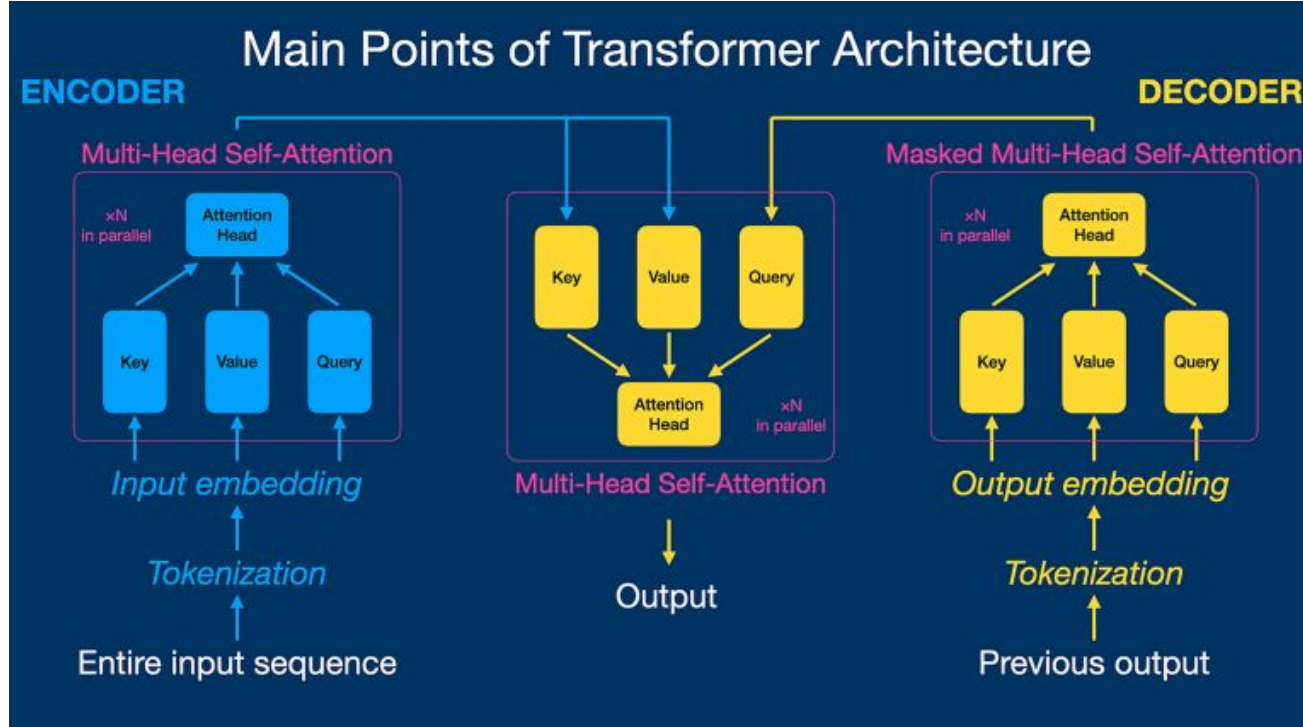
# Interação com IA em OpenAI e Gemini

LLM (Large Language Model) é um modelo de inteligência artificial treinado em enormes volumes de texto para entender e gerar linguagem natural. Ele aprende padrões estatísticos entre palavras e contextos, sendo capaz de responder perguntas, resumir textos, gerar código, traduzir e muito mais.

LLMs possuem “especialidades”, performando melhor ou pior que outros, dependendo do que é pedido. Alguns LLMs conhecidos são:

- ChatGPT (OpenAI): Produtividade geral, forte capacidade de raciocínio lógico.
- Gemini (Google): Integração com aplicativos do dia a dia e respostas rápidas.
- Claude (Anthropic): PLN, raciocínio e análise profunda de documentos longos.
- Nova (Amazon): Agentes inteligentes corporativos e soluções hospedadas na nuvem.
- Llama (Facebook): Execução local e stand-alone.

# Interação com IA em OpenAI e Gemini



Fonter: <https://blog.dsacademy.com.br/funcionamento-dos-llms-e-machine-unlearning>

# Interação com IA em OpenAI e Gemini

- FLASK (Servidor Web): **Flask** é um microframework Python para criar APIs HTTP. No contexto de IA, ele atua como intermediário entre o cliente e o modelo de linguagem;
- OPENAI (GPT as a Service): A biblioteca **openai** permite chamar modelos como GPT-4o e GPT-4 Turbo diretamente do Python;
- GEMINI (Google AI as a Service): A biblioteca **google-generativeai** permite chamar modelos como Gemini 1.5 Pro e Gemini 2.0 Flash;

| Camada       | Tecnologia                       | Papel                              |
|--------------|----------------------------------|------------------------------------|
| Linguagem    | Python                           | Base de tudo                       |
| Servidor web | Flask                            | Recebe e responde requisições HTTP |
| IA — OpenAI  | <code>openai</code>              | Chama GPT-4o, GPT-4 Turbo          |
| IA — Google  | <code>google-generativeai</code> | Chama Gemini 1.5 / 2.0             |

# Interação com IA em OpenAI e Gemini

As LLMs podem, de certa forma, serem vistas como “*AI as a Service*”, logo para fazer uso destes serviços precisamos de chaves de API.

Passo-a-passo para criar uma aplicação de IA com Python/Flask:

1. Gere sua API-Key no site do fornecedor da LLM;
2. Implemente o servidor Python com o Flask, expondo as rotas necessárias.  
Normalmente, usa-se ao menos uma rota POST para receber os tokens de entrada para a LLM e que serão usados como instrução/prompt pela IA;
3. Configure seu servidor com a instância do LLM, passando a chave de API. Processe os dados recebidos da requisição POST e envie-os como prompt para a LLM;
4. Obtenha a resposta da LLM, processe-a, e retorne os resultados processados para o cliente.

# Interação com IA em OpenAI e Gemini

## Configuração Padrão do OpenAI

```
#pip install -U openai

from openai import OpenAI

client = OpenAI(api_key="<API_KEY>")
response = client.responses.create(
    model="gpt-5.5",
    input="O que é a linguagem Python?"
)

print(response.output_text)
```

<https://platform.openai.com/api-keys>

## Configuração Padrão do Gemini

```
#pip install google-genai

from google import genai

client = genai.Client(api_key="<APY_KEY>")
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="O que é a linguagem Python?",
)

print(response.text)
```

<https://ai.google.dev/gemini-api/docs/api-key>



# Criação de Serviços e Integração com IA

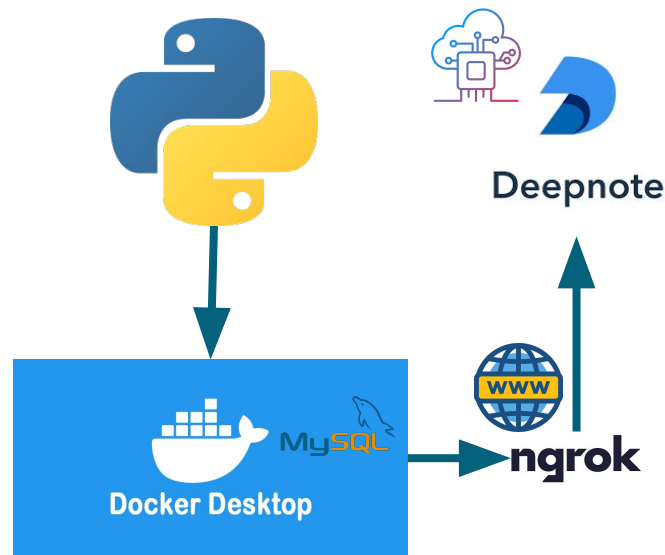
## PARTE 2

### **Implementação de Serviço Web com Flask**

# Implementação de Serviço Web com Flask

## Stack Laboratorial

- Conta Deepnote para conexão em nuvem e integração com camada de analytics;
- Tunneling com ngrok, para publicação de um servidor público Python;
- App Python, com receita dockerfile rodando e configs para expor porta 5000;
- Plataforma Docker instalada na máquina local, disponibilizando um contêiner python:3.11-slim;



**OPTIONAL**

# Implementação de Serviço Web com Flask

## Configuração do Servidor Python em Container Docker Desktop

1. Instalar o Docker Desktop v4.32.0 (157355):  
<https://www.docker.com/products/docker-desktop/>
2. Criar conta na HUB Docker:  
<https://login.docker.com/>
3. No prompt de Comando:
  - a. Criar uma ponte de rede para expor o contêiner:  
**`$ docker network create --driver bridge infnet-network`**
  - b. Construir o contêiner através da receita Dockerfile  
**`$ docker build -t python-rest-api .`**
  - c. Executar o contêiner com a aplicação, na porta 5000  
**`$ docker run -d -p 5000:5000 --name my-api-server python-rest-api`**

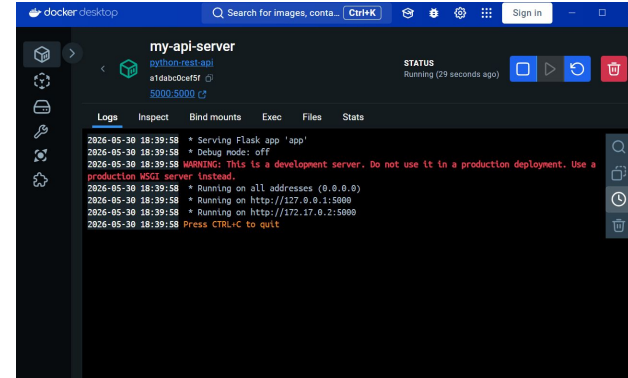
# Implementação de Serviço Web com Flask

## Configuração do Servidor Python em Container Docker Desktop

```
[+] Building 1.8s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 456B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.7s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 63B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb8409 0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                       0.0s
=> => exporting image                                             0.0s
=> => exporting layers                                             0.0s
=> => writing image sha256:d80clccf9ac24e8f14245e1642d7c51b1cd0ea0e0757720ce44cdfd6fb73cdd 0.0s
=> => naming to docker.io/library/python-rest-api                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qg1517sh66unwjcmo
xuhgb19z

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview
PS C:\Users\marce\Downloads\python\Python_para_Processamento_de_Dados_08_exe> |
```



| Search                              |             | Only running  |              |             |                          |                                                                   |
|-------------------------------------|-------------|---------------|--------------|-------------|--------------------------|-------------------------------------------------------------------|
| <input type="checkbox"/>            | Port(s)     | Name          | Container ID | Image       | C                        | Actions                                                           |
| <input checked="" type="checkbox"/> | 5000:5000 ↗ | my-api-server | 28291f87a2f2 | python-rest | <input type="checkbox"/> | <input type="button" value="⋮"/> <input type="button" value="🗑"/> |

**OPTIONAL**

# Implementação de Serviço Web com Flask

## Configuração do ngrok e tunneling para a rede pública

```
C:\Program Files\WindowsAp x + v
ngrok (Ctrl+C to quit)
Request early access to new features: https://dashboard.ngrok.com/early-access
Session Status online
Account hama.marcelo.ti@gmail.com (Plan: Free)
Version 3.39.1-msix-stable
Region South America (sa)
Latency 25ms
Web Interface http://127.0.0.1:4040
Forwarding https://ad4f-2804-14d-78d0-473d-e9ce-89-f06f-37da.ngrok-free.app -> http://localhost:5000
Connections
  ttl   opn   rt1   rt5   p50   p90
   0     0    0.00  0.00  0.00  0.00
```

1. Webpage do ngrok e instruções (necessário configurar chave de API/token):  
<https://ngrok.com/download/>
2. No prompt de Comando:
  - a. Exposição do contêiner Python server através da porta http 5000:  
***\$ ngrok http 5000***
  - b. Anote a URL de forwarding e use-a no Deepnote.

# Implementação de Serviço Web com Flask

## Aplicação Servidor Teste

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Persistencia de dados em memoria
data_store = [
    {"id": 1, "name": "Item One"},
    {"id": 2, "name": "Item Two"}
]

@app.route('/items', methods=['GET'])
def get_items():
    return jsonify({"items": data_store}), 200
```

```
@app.route('/items', methods=['POST'])
def add_item():
    new_item = request.get_json()
    if not new_item or 'name' not in new_item:
        return jsonify({"error": "Bad Request",
            "message": "Missing 'name' field"}), 400

    new_item['id'] = len(data_store) + 1
    data_store.append(new_item)
    return jsonify(new_item), 201

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

# Implementação de Serviço Web com Flask

## Aplicação Cliente Teste

```
import requests
```

```
BASE_URL = "<SERVER_URL>/items"
```

```
# 1. Test GET request
```

```
print("--- Testing GET ---")
```

```
response = requests.get(BASE_URL)
```

```
print(f"Status Code: {response.status_code}")
```

```
print(f"Response JSON: {response.json()}\n")
```

```
# 2. Test POST request
```

```
print("--- Testing POST ---")
```

```
payload = {"name": "Item Three"}
```

```
response = requests.post(BASE_URL, json=payload)
```

```
print(f"Status Code: {response.status_code}")
```

```
print(f"Response JSON: {response.json()}\n")
```

```
# 3. Test GET again to confirm item was added
```

```
print("--- Testing GET (After POST) ---")
```

```
response = requests.get(BASE_URL)
```

```
print(f"Response JSON: {response.json()}\n")
```

# Implementação de Serviço Web com Flask

## Configurações do Flask

# 0 Flask armazena configurações em app.config,  
# um dicionário alimentado de várias formas.

```
from flask import Flask  
app = Flask(__name__)
```

# 1 – diretamente no código  
app.config["DEBUG"] = True  
app.config["SECRET\_KEY"] = "minha-chave"

# 2 – update() para múltiplas chaves de uma vez  
app.config.update(  
 DEBUG = True,  
 SECRET\_KEY = "minha-chave",  
 MAX\_CONTENT\_LENGTH = 16 \* 1024 \* 1024  
)

# 3 – a partir de variáveis de ambiente

```
import os  
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY",  
    "fallback-dev")
```

# 4 – arquivo .env com python-dotenv

```
from dotenv import load_dotenv  
load_dotenv()  
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY")
```

# 5 – a partir de um arquivo Python externo

```
app.config.from_pyfile("config.py")
```

# 6 – a partir de um objeto/classe

```
class Config:  
    DEBUG = False  
    SECRET_KEY = "chave-producao"  
app.config.from_object(Config)
```



# Criação de Serviços e Integração com IA

## PARTE 3

### **Endpoints com JSON Object em Flask**

# Endpoints com JSON Object em Flask

## Conceito de API e Endpoint

Um endpoint é um ponto de acesso de uma API — uma URL específica que aceita requisições HTTP e retorna uma resposta. É a combinação de uma URL com um método HTTP (GET, POST, PUT, DELETE etc.).

```
# Query-String GET /buscar?nome=ana&cidade=sp
@app.route("/buscar", methods=["GET"])
def buscar():
    nome = request.args.get("nome", "")
    cidade = request.args.get("cidade", "")
    return jsonify({
        "nome": nome,
        "cidade": cidade
    })
```

# endpoint mais simples – GET

```
@app.route("/")
def home():
    return jsonify({"status": "ok"})
```

# especificando o método HTTP

```
@app.route("/ping", methods=["GET"])
def ping():
    return jsonify({"mensagem": "pong"})
```

# aceitando múltiplos métodos

```
@app.route("/dados", methods=["GET", "POST"])
def dados():
    if request.method == "GET":
        return jsonify({"acao": "listando dados"})
    if request.method == "POST":
        return jsonify({"acao": "criando dado"})
```

# Endpoints com JSON Object em Flask

Python é uma linguagem com alto poder de integração que disponibiliza a biblioteca Flask para expor serviços como uma API web. Bibliotecas de IA (OpenAI e Gemini) fornecem as capacidades de linguagem e IA. Juntos formam o padrão mais comum para construir APIs de IA prontas para produção.

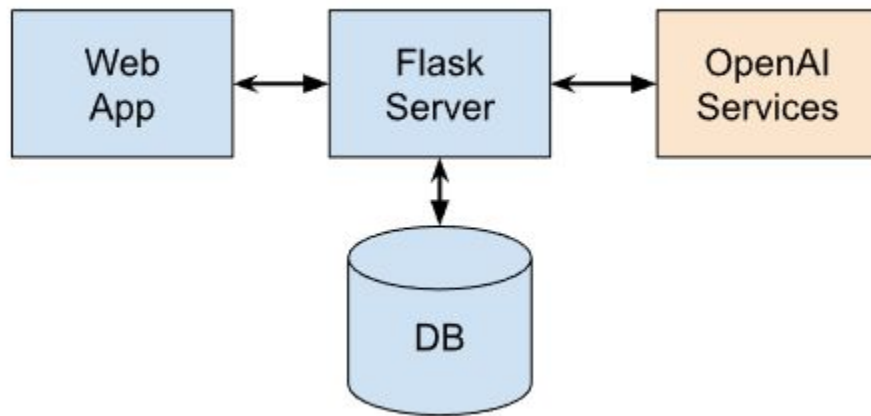


Diagrama padrão de uma aplicação Python/Flask utilizando serviços de IA.

# Endpoints com JSON Object em Flask

## OpenAI: Servidor e Cliente

```
client = OpenAI(api_key="<API_KEY>")
@app.route("/chat_with_openai", methods=["POST"])
def chat_with_openai():
    body = request.get_json()
    pergunta = body.get("pergunta", "")

    resposta = client.responses.create(
        model="gpt-5.5",
        input=pergunta
    )

    return jsonify({
        "resposta": resposta.output_text
    })
```

```
import requests

BASE = "<SERVER_URL>"

r = requests.post(f"{BASE}/chat_with_openai",
    json={
        "pergunta": "Quais as características do
Python?"
    })

print(r.status_code)
print(r.json()["resposta"])
```

# Endpoints com JSON Object em Flask

## Gemini: Servidor e Cliente

```
client = genai.Client(api_key="<API_KEY>")
@app.route("/chat_with_gemini", methods=["POST"])
def chat_with_gemini():
    body = request.get_json()
    pergunta = body.get("pergunta", "")

    resposta = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=pergunta
    )

    return jsonify({
        "resposta": resposta.text
    })
```

```
import requests

BASE = "<SERVER_URL>"

r = requests.post(f"{BASE}/chat_with_gemini",
    json={
        "pergunta": "Quais as características do
Python?"
    })

print(r.status_code)
print(r.json()["resposta"])
```

# Endpoints com JSON Object em Flask

## Claude: Servidor e Cliente

```
client = Anthropic(api_key="<API_KEY>")
@app.route("/chat_with_claude", methods=["POST"])
def chat_with_claude():
    body = request.get_json()
    pergunta = body.get("pergunta", "")
    resposta = client.messages.create(
        model = "claude-opus-4-6",
        max_tokens = 1024,
        messages = [{"role": "user", "content":
pergunta}]
    )
    return jsonify({
        "resposta": resposta.content[0].text
    })
```

```
import requests

BASE = "<SERVER_URL>"

r = requests.post(f"{BASE}/chat_with_claude",
    json={
        "pergunta": "Quais as características do
Python?"
    })

print(r.status_code)
print(r.json()["resposta"])
```

# Endpoints com JSON Object em Flask

## Servidor com Todos os LLMs

```
import os
from flask import Flask, request, jsonify
from anthropic import Anthropic
from google import genai
from openai import OpenAI

app = Flask(__name__)

# Código com as implementações das
# funções de chat com os modelos...

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

# Criação de Serviços e Integração com IA

## PARTE 4

### **Consumo de Endpoint com Requisição Web GET**



# Consumo de Endpoint com Requisição Web GET

## Consumo de Serviços

Consumir um endpoint significa fazer uma requisição HTTP para uma URL de uma API e usar a resposta retornada. Quem cria o endpoint é o servidor (Flask, FastAPI etc.). Quem consome é o cliente, que pode ser um app mobile, um browser, outro serviço, ou um script Python. Criar é expor uma funcionalidade via URL. Consumir é chamar essa URL e usar o que ela retorna.

```
# SERVIDOR - cria o endpoint (Flask)
@app.route("/soma", methods=["POST"])
def soma():
    body = request.get_json()
    resultado = body["a"] + body["b"]
    return jsonify({"resultado": resultado})
```

```
# CLIENTE - consome o endpoint (requests)
r = requests.post("http://localhost:5000/soma",
                  json={"a": 3, "b": 7})

print(r.json()["resultado"])    # 10
```

# Consumo de Endpoint com Requisição Web GET

## Chatbot Multi-Model

```
import requests
```

```
BASE = "SERVER_URL"
```

```
LLMS = (  
    "claude",  
    "gemini",  
    "openai"  
)
```

```
client = "claude"
```

```
while True:
```

```
    pergunta = input("Digite sua pergunta: ")
```

```
    if pergunta == "sair":
```

```
        break
```

```
    if pergunta == "trocar":
```

```
        c = input("Digite o nome do cliente: ")
```

```
        client = c if c in LLMS else print("LLM inválido") or client
```

```
        continue
```

```
    try:
```

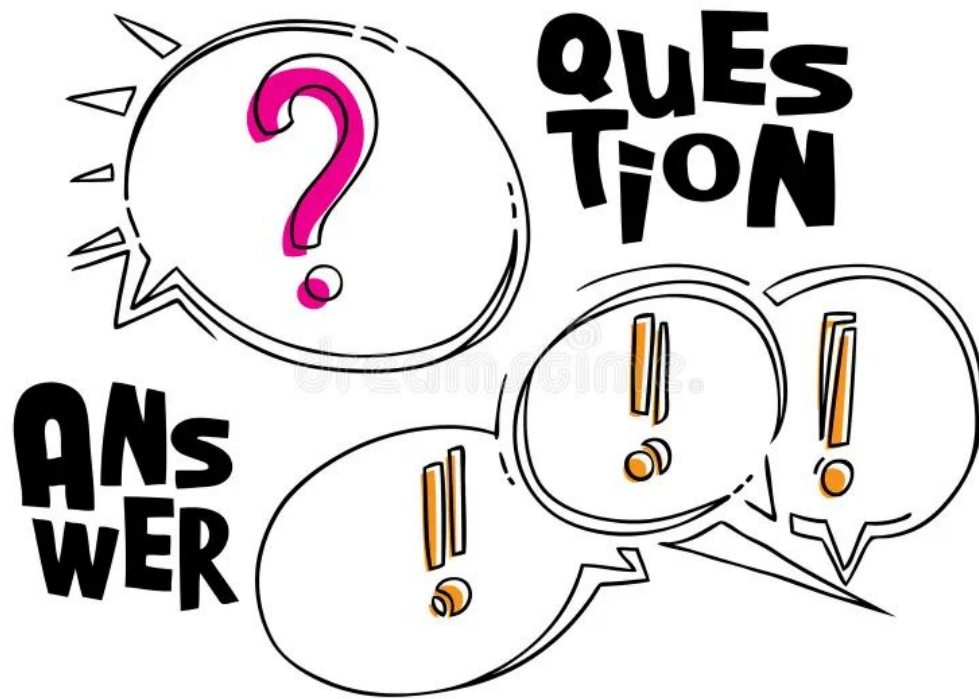
```
        r = requests.post(  
            f"{BASE}/chat_with_{client}",  
            json={"pergunta": pergunta})
```

```
        print(r.json()["resposta"])
```

```
    except Exception as e:
```

```
        print(f"Erro ao conectar com o servidor: {e}")
```

# Dúvidas e Perguntas



# Referências Bibliográficas

- Get Programming, Ana Bell, Manning Publications;
- Introducing Python, Bill Lubanovic, O'Reilly Media, Inc;
- Learn to Code by Solving Problems, Daniel Zingaro, No Starch Press queue;
- Python Crash Course, Eric Matthes, No Starch Press;
- Streamlining Your Research Laboratory with Python, Mark F. Russo, William Neil, Wiley;
- The Quick Python Book, Naomi Ceder, Manning Publications;
- <https://deepnote.com/docs/incoming-connections>;
- <https://ai.google.dev/gemini-api/docs>;
- <https://developers.openai.com/api/docs/libraries>.